

How Linear Implemented Multi-Region Support For Customers



BYTEBYTEGO

SEP 15, 2025



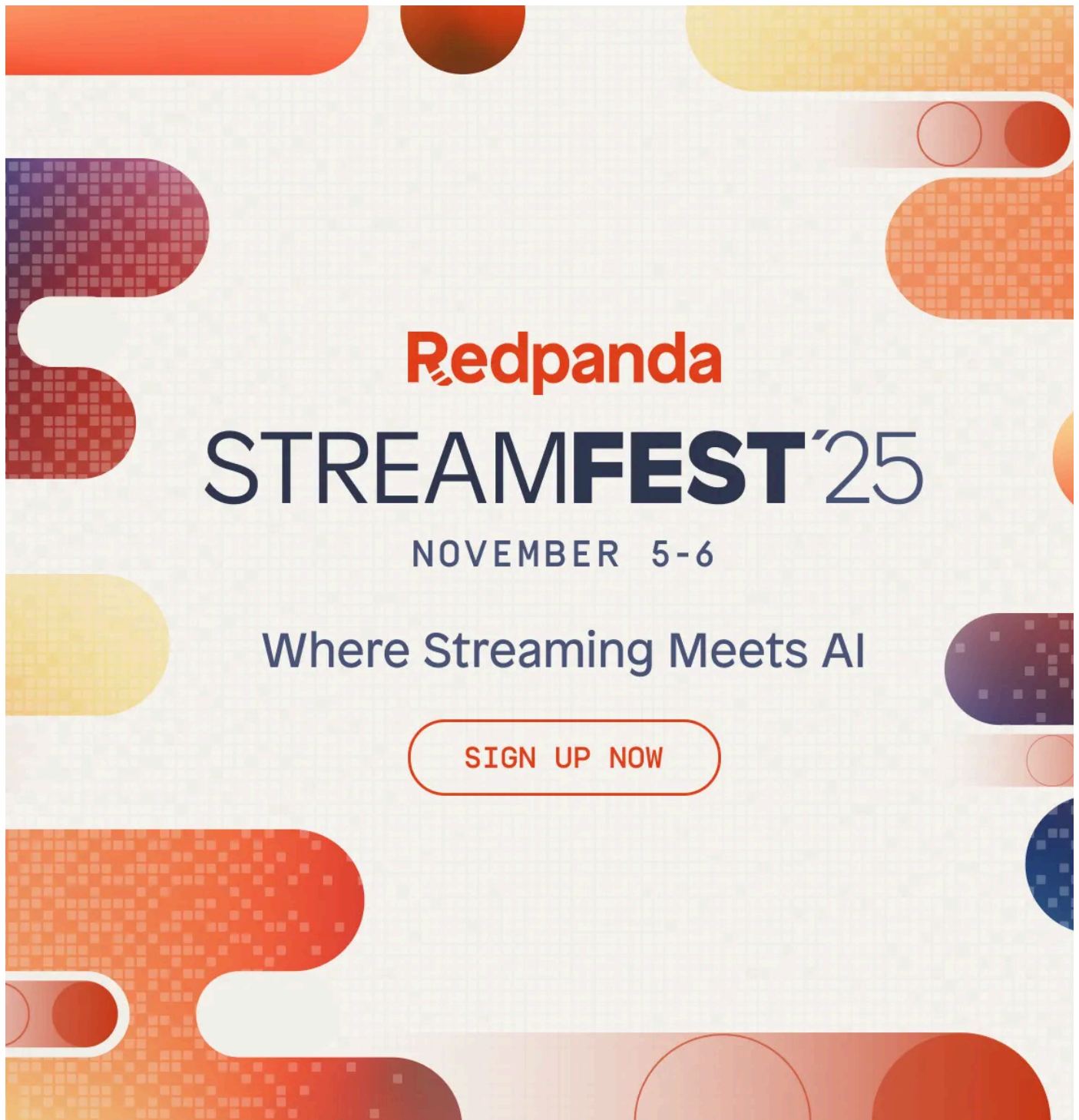
229



4

Share

[Streamfest 2025: Streaming Meets AI \(Sponsored\)](#)



Join us November 5–6 for **Redpanda Streamfest**, a two-day online event dedicated to streaming data technologies for agentic and data-intensive applications. Learn how to build scalable, reliable, and secure data pipelines through technical sessions, live demos, and hands-on workshops. Sessions include keynotes from industry leaders, real-world case studies, and tutorials on next-gen connectors for AI use cases.

Discover why a streaming data foundation is essential for LLM-powered applications, how to simplify architectures, and new approaches to cost-effective storage. Connect with experts, sharpen your skills, and get ready to unlock the full potential of AI with streaming.

Disclaimer: The details in this post have been derived from the official documentation shared online by the Linear Engineering Team. All credit for the technical details goes to the Linear Engineering Team. The links to the original articles and sources are present in the reference section at the end of the post. We've attempted to analyze the details and provide our input about them. If you find any inaccuracies or omissions, please leave a comment, and we will do our best to fix them.

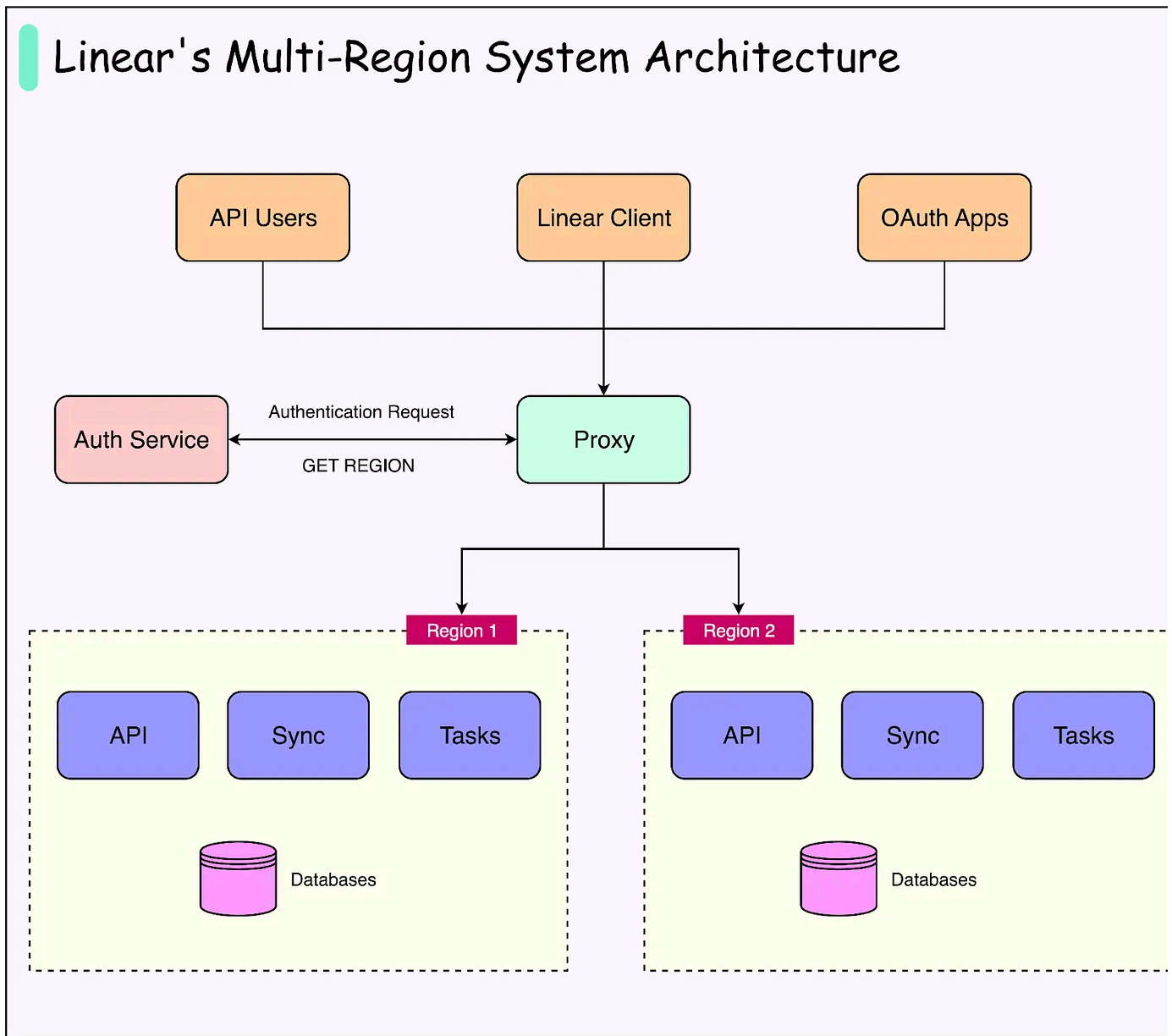
Linear represents a new generation of project management platforms specifically designed for modern software teams.

Founded in 2019, the company has built its reputation on delivering exceptional speed and developer experience, distinguishing itself in a crowded market dominated by established players like Jira and Asana.

What sets Linear apart is its focus on performance, achieving sub-50ms interaction times that make the application feel instantaneous. The platform embraces a keyboard-first design philosophy, allowing developers to navigate and manage their work without reaching for the mouse, a feature that resonates strongly with its technical user base.

The modern SaaS landscape presents a fundamental challenge that every growing platform must eventually face: how to serve a global customer base while respecting regional data requirements and maintaining optimal performance.

In this article, we look at how Linear implemented multi-region support for its customers. We will explore the architecture they built, along with the technical implementation details.



The Need for Multi-Region Support

The decision to implement multi-region support at Linear wasn't made in a vacuum but emerged from concrete business pressures and technical foresight. There were couple of reasons:

- **Compliance:** The most immediate driver came from the European market, where larger enterprises expressed clear preferences for hosting their data within

Europe. This was due to the need for GDPR compliance and internal data governance policies.

- **Technical:** The primary technical concern centered on the eventual scaling limit of their PostgreSQL infrastructure. While their single-region deployment in Google Cloud's us-east-1 was serving them well, the team understood that continuing to scale vertically by simply adding more resources to a single database instance would eventually hit hard limits. By implementing multi-region support early, they created a horizontal scaling path that would allow them to distribute workspaces across multiple independent deployments, each with its own database infrastructure.
- **Competitive Advantage:** The implementation of multi-region support also positioned Linear more favorably in the competitive project management space. By offering European data hosting, Linear could compete more effectively for enterprise contracts against established players who might not offer similar regional options.

The Architecture Design Principles

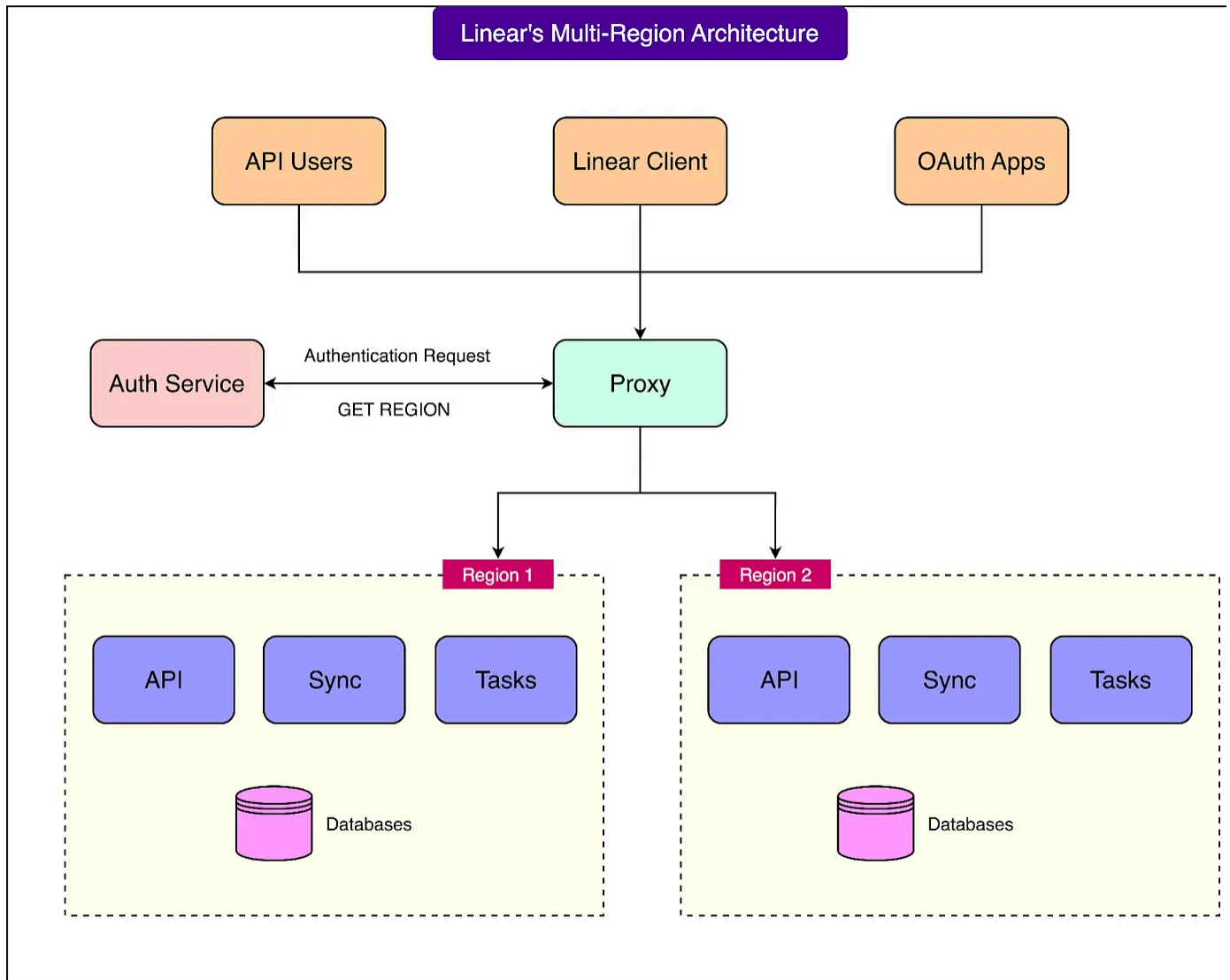
The multi-region architecture Linear implemented follows four strict requirements that shaped every technical decision in the system.

- **Invisible to Users:** This meant maintaining single domains (linear.app and api.linear.app) regardless of data location. This constraint eliminated the simple approach of region-specific subdomains like eu.linear.app, which would have pushed complexity onto users and broken existing integrations. Instead, the routing logic lives entirely within the infrastructure layer.
- **Developer Simplicity:** This meant that engineers writing application features shouldn't need to consider multi-region logic in their code. This constraint influenced numerous implementation details, from the choice to replicate entire deployments rather than shard databases to the decision to handle all

synchronization through background tasks rather than synchronous cross-region calls.

- **Feature Parity:** Every Linear feature, integration, and API endpoint must function identically regardless of which region hosts a workspace. This eliminated the possibility of region-specific feature flags or degraded functionality, which would have simplified the implementation but compromised the user experience.
- **Full Regional Isolation:** This meant that each region operates independently. A database failure, deployment issue, or traffic spike in one region cannot affect the other. This isolation provides both reliability benefits and operational flexibility. Each region can be scaled, deployed, and maintained independently based on its specific requirements.

The following architecture diagram reveals a three-tier structure.



User-facing clients (API users, the Linear web client, and OAuth applications) all connect to a central proxy layer. This proxy communicates with an authentication service to determine request routing, then forwards traffic to one of two regional deployments. Each region contains a complete Linear stack: API servers, sync engine, background task processors, and databases.

The proxy layer, implemented using Cloudflare Workers, serves as the routing brain of the system. When a request arrives, the proxy extracts authentication information, queries the auth service for the workspace's region, and obtains a signed JWT, then forwards the request to the appropriate regional deployment. This happens on every

request, though caching mechanisms reduce the overhead for frequent requests from the same client.

The resulting architecture trades implementation complexity for operational benefit and user experience. Rather than distributing complexity across the application or pushing it onto users, Linear concentrated it within well-defined infrastructure components—primarily the proxy and authentication service.

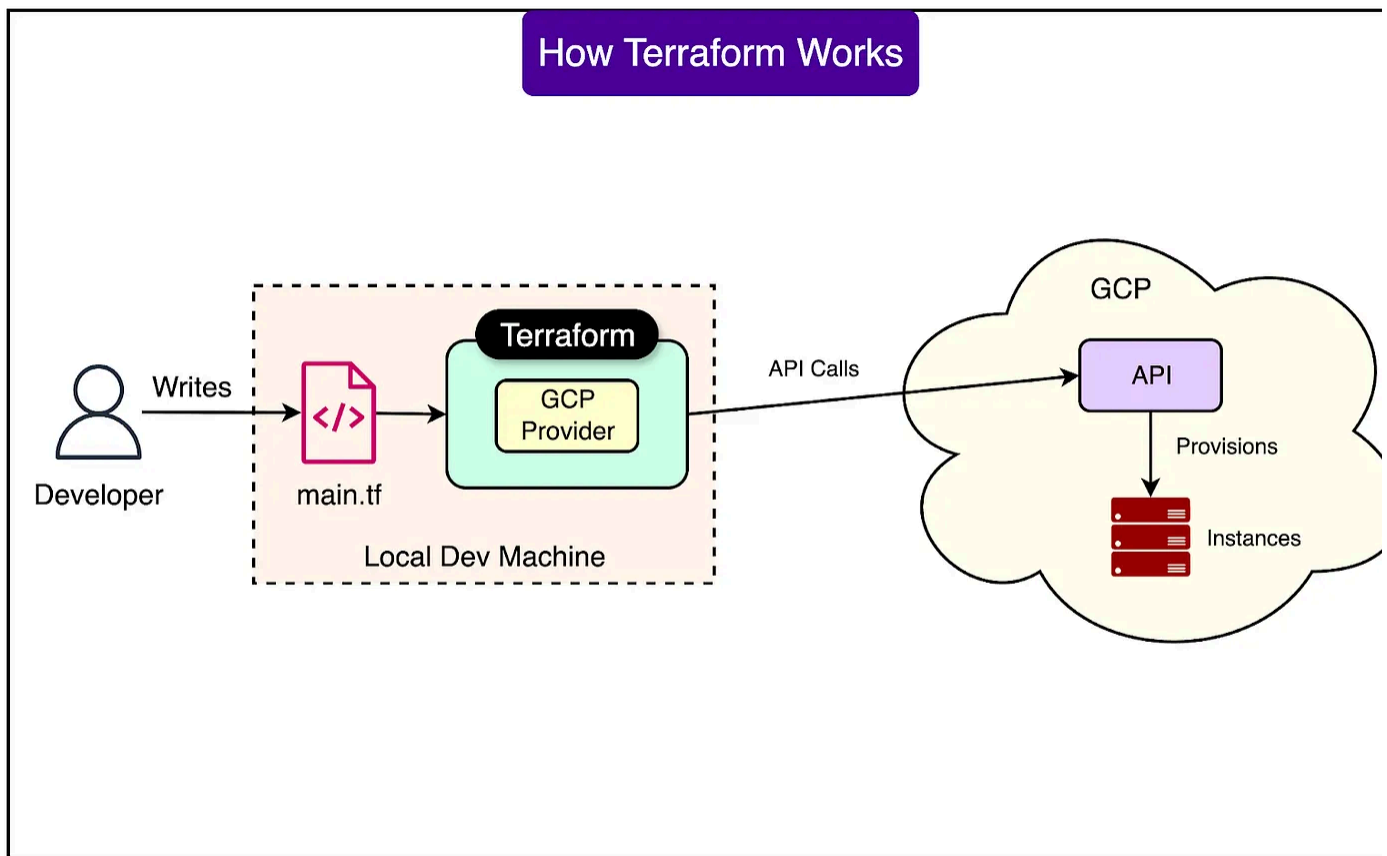
Technical Implementation Phases

There were three main phases to the technical implementation:

1 - Infrastructure as Code Transformation

Before implementing multi-region support, Linear's infrastructure existed as manually configured resources in Google Cloud Platform. While functional for a single-region deployment, this approach wouldn't scale to managing multiple regional deployments. The manual configuration would have required duplicating every setup step for each new region, creating opportunities for configuration drift and human error.

The transformation began with Google Cloud's Terraform export tooling, which generated Terraform configurations from the existing infrastructure. This automated export provided a comprehensive snapshot of their us-east-1 deployment, but the raw export required significant cleanup. The team removed resources that weren't essential for the main application, particularly global resources that wouldn't need regional replication and resources that hadn't been manually created originally.



The critical work involved refactoring these Terraform resources into reusable modules. Each module was designed to accept region as a variable parameter, along with region-specific configurations for credentials and secrets. This modular approach transformed infrastructure deployment from a manual process into a parameterized repeatable operation. Spinning up a new region became a matter of instantiating the modules with appropriate regional values rather than recreating infrastructure from scratch.

The team also built a staging environment using these Terraform modules, which served multiple purposes.

- Validated that the infrastructure-as-code accurately replicated their production environment.
- Provided a safe space for testing infrastructure changes before production deployment, and crucially

- Provided an environment for testing the proxy's routing logic that would direct traffic between regions.

2 - Authentication Service Architecture

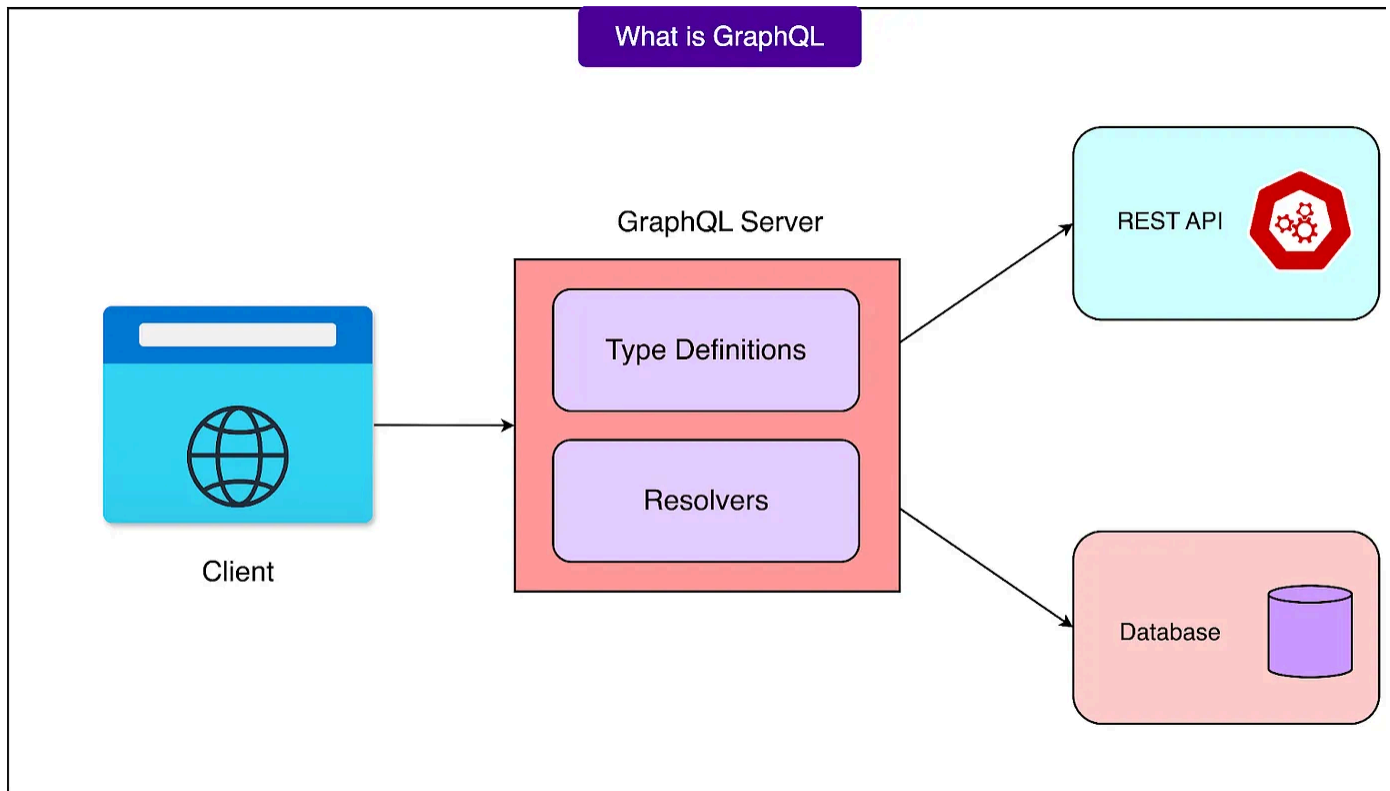
The authentication service extraction represented the most complex phase of Linear's multi-region implementation, touching large portions of their codebase.

The extraction followed a gradual approach designed to minimize risk. Initially, while still operating in a single region, the new authentication service shared a database with the main backend service in its US region. This co-location allowed the team to develop and test the extraction logic without immediately dealing with network latency.

Once the extraction logic was functionally complete, they implemented strict separation at the database level. Tables were split into distinct schemas with database-level permission boundaries—the authentication service couldn't read or write regional data, and regional services couldn't directly access authentication tables. This hard boundary, enforced by PostgreSQL permissions rather than application code, guaranteed that the architectural separation couldn't be accidentally violated by a coding error.

Some tables required splitting between the two services. For example, workspace configuration contained both authentication-relevant settings and application-specific data. The solution involved maintaining parallel tables with a one-to-one relationship for shared fields, requiring careful synchronization to maintain consistency.

Linear adopted a one-way data flow pattern that simplified the overall architecture. Regional services could call the authentication service directly through a GraphQL API, but the authentication service never made synchronous calls to regional services. When the authentication service needed regional actions, it scheduled background tasks using Google Pub/Sub's one-to-many pattern, broadcasting tasks to all regions.



This design choice meant the authentication service only handled HTTP requests without needing its own background task runner, simplifying deployment and operations. The choice of GraphQL for internal service communication leveraged Linear's existing investment in GraphQL tooling from their public API. Using Zeus to generate type-safe clients eliminated many potential integration errors and accelerated development by reusing familiar patterns and tools.

Three distinct patterns emerged for maintaining data consistency between services

- Record creation always began in the authentication service to ensure global uniqueness constraints (like workspace URL keys) were enforced before creating regional records. The authentication service would return an ID that regional services used to create corresponding records,
- Updates used Linear's existing sync engine infrastructure. When a regional service updated a shared record, it would asynchronously propagate changes to

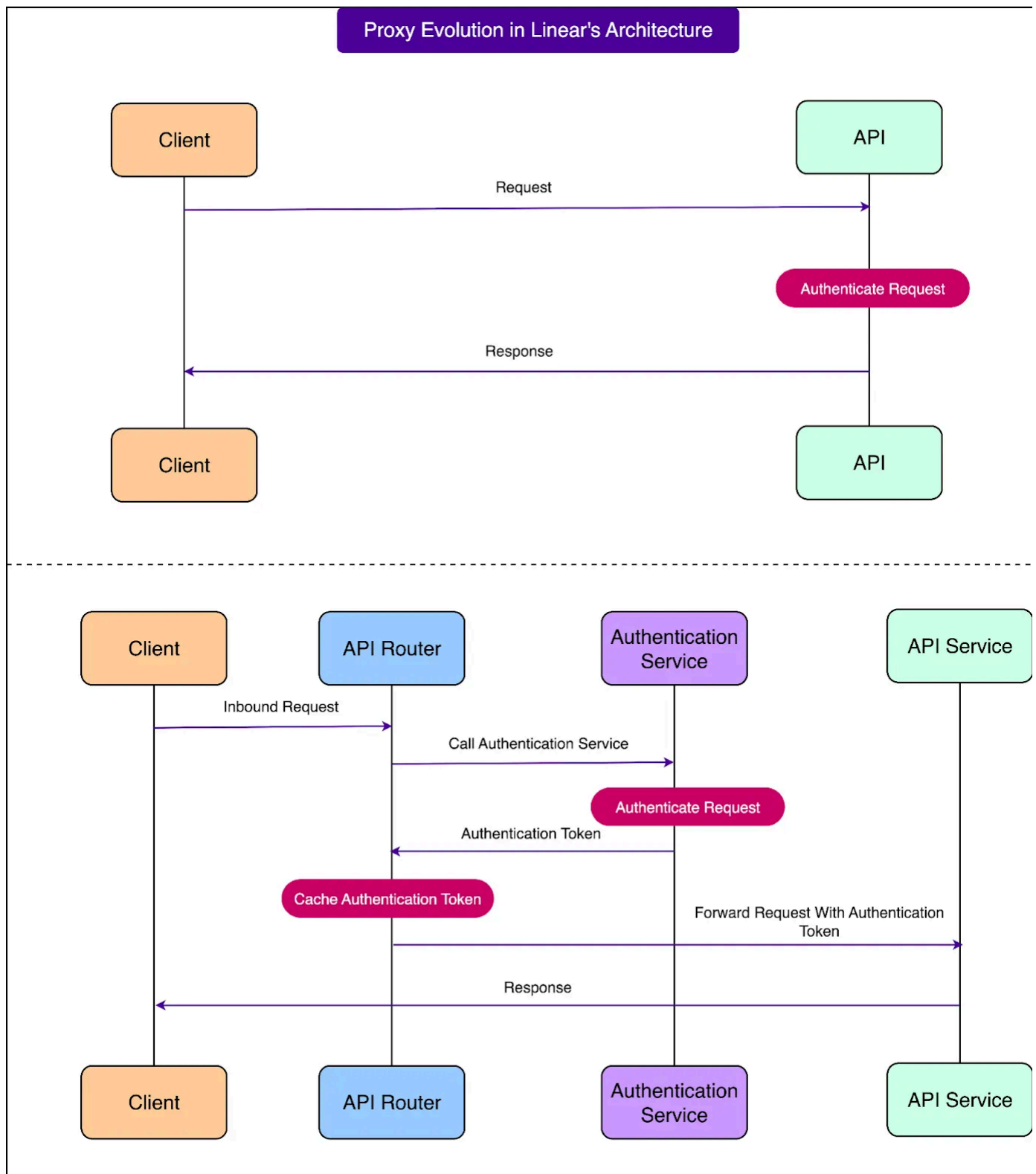
the authentication service. This approach kept the update path simple for developers. They just updated the records normally in the regional service.

- Deletion worked similarly to creation, with additional complexity from foreign key cascades. PostgreSQL triggers created audit logs of deleted records, capturing deletions that occurred due to cascading from related tables.

3 - Request Routing Layer

The final piece of Linear's multi-region architecture involved implementing a proxy layer to route requests to the appropriate regional deployment. Since Linear already used Cloudflare Workers extensively, extending their use for request routing was a natural choice.

See the diagram below:



The proxy worker's core responsibility is straightforward but critical.

For each incoming request, it extracts authentication information (from cookies, headers, or API tokens), makes a call to the authentication service to determine both the target region and obtain a signed JWT, then forwards the request to the appropriate regional deployment with the pre-signed header attached.

The JWT signing mechanism serves dual purposes. It validates that requests have been properly authenticated and authorized by the central authentication service, while also carrying metadata about the user and workspace context. This eliminates the need for regional services to make their own authentication calls, reducing latency and system complexity.

To optimize performance, the Cloudflare Worker implements sophisticated caching of authentication signatures.

When the same client makes frequent requests, the worker can serve cached authentication tokens without making repeated round-trip to the authentication service. The caching strategy had to balance performance with security. Tokens are cached for limited periods and include enough context to prevent cache poisoning attacks while still providing meaningful performance benefits for active users making multiple rapid requests.

Linear's real-time sync functionality relies heavily on WebSockets, making their efficient handling crucial for the multi-region architecture. The team leveraged an important Cloudflare Workers optimization: when a worker returns a fetch request without modifying the response body, Cloudflare automatically hands off to a more efficient code path.

This optimization is particularly valuable for long-lived connections. Rather than keeping a worker instance active for the entire duration of a WebSocket connection, the handoff mechanism allows the worker to complete its routing decision and then step aside, letting Cloudflare's infrastructure efficiently proxy the established connection to the regional deployment.

During the implementation, the team kept fallback mechanisms in place. While building out the proxy layer, the API service could still authenticate requests direct they weren't pre-signed by the proxy.

Conclusion

Linear's multi-region implementation demonstrates that supporting geographical distribution doesn't require sacrificing simplicity or performance.

By concentrating complexity within well-defined infrastructure components (the proxy layer and authentication service) the architecture shields both users and developers from the underlying regional complexity. The extensive behind-the-scenes work touched sensitive authentication logic throughout the codebase, yet careful planning meant most bugs remained invisible to users through strategic use of fallbacks and gradual rollouts.

The implementation now enables workspace creation in Linear's European region with full feature parity, automatically selecting the default region based on the user timezone while preserving choice. The architecture positions Linear for future expansion, with the framework in place to add additional regions as needed. The team plans to extend this capability further by supporting workspace migration between regions, allowing existing customers to relocate their data as requirements change.

References:

- [How we built multi-region support for Linear](#)
- [Terraform Modules](#)
- [Introduction to GraphQL](#)

Help us Make ByteByteGo Newsletter Better

TL:DR: Take this 2-minute survey so I can learn more about who you are,. what you and how I can improve ByteByteGo

SPONSOR US

Get your product in front of more than 1,000,000 tech professionals.

Our newsletter puts your products and services directly in front of an audience that matters - hundreds of thousands of engineering leaders and senior engineers - who have influence over significant tech decisions and big purchases.

Space Fills Up Fast - Reserve Today

Ad spots typically sell out about 4 weeks in advance. To ensure your ad reaches this influential audience, reserve your space now by emailing sponsorship@bytebytego.com.



229 Likes • 4 Restacks

Discussion about this post

Comments Restacks



Write a comment...

© 2026 ByteByteGo · [Privacy](#) · [Terms](#) · [Collection notice](#)
[Substack](#) is the home for great culture